

Roteiro Teórico da Apresentação

Rede Social Simplificada — Projeto Final, Tema 04

Como usar este documento: ele acompanha os 14 slides de `slides.html` e o roteiro de demonstração ao vivo em `roteiro_demo.md`. Enquanto aquele roteiro foca no *o que digitar e o que vai aparecer na tela*, este aqui é o material de apoio teórico — os conceitos de estruturas de dados e algoritmos por trás de cada slide, resumidos para serem falados em 30–60 segundos cada. É o conteúdo que justifica, na introdução do relatório, por que cada estrutura foi escolhida.

Sumário

1	Introdução: o que é um TAD e por que isso importa	1
2	Arquitetura em fachada	1
3	As seis estruturas de dados obrigatórias	2
3.1	Árvore AVL — índice por id	2
3.2	Tabela hash — índice por login e por nome	2
3.3	Lista encadeada — publicações e adjacência	3
3.4	Fila — motor da busca em largura	3
3.5	Pilha — histórico de atividades	3
3.6	Grafo — rede de amigos	3
4	Busca em largura (BFS): o algoritmo por trás de duas funcionalidades	4
5	Requisitos não funcionais: persistência e tratamento de erros	4
6	Testes: unitário vs. integração	5
7	Tabela-resumo: estrutura, operação-chave e complexidade	5
	Fechamento	5

1 Introdução: o que é um TAD e por que isso importa

[Slide 1–2]

Um **Tipo Abstrato de Dados (TAD)** é a definição de um conjunto de dados junto com as operações permitidas sobre eles, *sem* amarrar essa definição a uma implementação específica. Uma pilha, por exemplo, é definida pelo comportamento “último a entrar, primeiro a sair” e pelas operações **empilhar/desempilhar** — não importa se por baixo dos panos ela usa um vetor ou uma lista encadeada. Essa separação entre *interface* e *implementação* é o que permite trocar a forma como uma estrutura funciona por dentro sem quebrar quem a utiliza por fora — e é

exatamente por isso que o projeto exige cada estrutura como um par independente `arquivo.h` (a interface, o contrato) e `arquivo.c` (a implementação).

A escolha de qual estrutura usar em cada parte do sistema não é arbitrária: depende de **qual operação precisa ser rápida**. Medimos isso com **notação O grande (Big-O)**, que descreve como o tempo (ou espaço) necessário cresce conforme o tamanho da entrada n aumenta:

- $O(1)$ — tempo constante, independe de n (ex.: acessar um bucket de hash).
- $O(\log n)$ — cresce devagar; dobrar n só soma um passo a mais (ex.: buscar em uma árvore balanceada).
- $O(n)$ — cresce proporcional a n (ex.: percorrer uma lista inteira).
- $O(V + E)$ — para grafos, cresce com o número de vértices V mais o de arestas E (ex.: BFS).

O projeto usa seis TADs justamente porque cada funcionalidade obrigatória tem um perfil de acesso diferente: buscar por id pede $O(\log n)$ garantido, validar login duplicado pede $O(1)$ médio, listar publicações pede inserção $O(1)$, e assim por diante — é o que as próximas seções detalham.

2 Arquitetura em fachada

[Slide 3]

O sistema segue o **padrão de projeto Fachada (Facade)**: um único módulo (`rede_social.c`) concentra todo o estado (a árvore, as duas tabelas hash, o grafo, a pilha de histórico) e expõe uma função de alto nível por funcionalidade obrigatória (`rs_cadastrar_usuario`, `rs_adicionar_amizade`, etc.). O `main.c` não sabe — e não precisa saber — como cada operação é resolvida por dentro; ele só faz o loop do menu e chama a fachada.

Essa separação entre *lógica de domínio* e *interface de usuário* é o que permite testar todas as regras de negócio (duplicidade de login, amizade repetida, ordenação de sugestões) sem precisar simular um terminal — é exatamente o que `make test` faz.

3 As seis estruturas de dados obrigatórias

[Slide 4]

Esta é a seção central: para cada estrutura, o conceito, como ela resolve colisões/desbalanceamento (quando aplicável) e por que ela foi a escolha certa para aquele papel no sistema.

3.1 Árvore AVL — índice por id

Uma **árvore binária de busca (BST)** organiza chaves de forma que, em cada nó, tudo à esquerda é menor e tudo à direita é maior. Isso permite busca, inserção e remoção em tempo proporcional à **altura** da árvore. O problema: se os valores chegarem em ordem crescente (ou decrescente), uma BST comum degenera em uma lista encadeada disfarçada — altura $O(n)$, e a promessa de busca rápida desaparece exatamente no pior momento.

A **árvore AVL** (Adelson-Velsky e Landis, 1962) resolve isso acrescentando uma invariante: para todo nó, a diferença de altura entre as subárvores esquerda e direita (o *fator de balanceamento*) nunca pode ser maior que 1. Sempre que uma inserção quebra essa invariante em algum ancestral, uma **rotação** (simples ou dupla) reorganiza um pequeno trecho da árvore para restaurar o balanceamento em tempo $O(1)$. O resultado é uma altura sempre $O(\log n)$, garantida — não apenas no caso médio.

Por que aqui: índice `id` -> `Usuario*`. Ids são gerados de forma sequencial (1, 2, 3, ...) — exatamente o padrão de entrada que quebraria uma BST comum. A AVL garante busca por id em $O(\log n)$ mesmo nesse cenário adversarial.

3.2 Tabela hash — índice por login e por nome

Uma **tabela hash** usa uma **função hash** $h(chave)$ para calcular diretamente a posição (bucket) de um vetor onde aquele dado deve estar, trocando a busca por comparações sucessivas (como numa árvore) por um cálculo direto — em média, $O(1)$. Uma boa função hash distribui as chaves uniformemente pelos buckets; o projeto usa a **djb2**, um hash multiplicativo simples e rápido para strings.

Como duas chaves diferentes podem calcular o mesmo bucket (**colisão**), é preciso uma estratégia de resolução. O projeto usa **encadeamento separado**: cada bucket é, ele mesmo, uma lista encadeada de entradas — colisões viram apenas uma lista um pouco mais longa para percorrer, sem invalidar outras posições do vetor.

O desempenho $O(1)$ depende do **fator de carga** (número de elementos dividido pelo número de buckets) se manter baixo. Quando ultrapassa um limiar (aqui, 0,75), a tabela faz um **rehash**: cria um vetor maior e redistribui todas as chaves — mantendo o custo médio amortizado em $O(1)$ mesmo com a tabela crescendo indefinidamente.

Por que aqui: duas instâncias independentes. Uma indexa por **login** (chave única, usada para barrar cadastro duplicado em $O(1)$); outra indexa por **nome** (chave repetível, cada bucket acumula uma lista de usuários com aquele nome, usada na busca por nome).

3.3 Lista encadeada — publicações e adjacência

Uma **lista encadeada** é uma sequência de nós onde cada nó guarda um dado e um ponteiro para o próximo — ao contrário de um vetor, os elementos não precisam ocupar posições contíguas de memória. Isso torna a **inserção no início** uma operação $O(1)$: basta criar o novo nó apontando para o antigo início, sem deslocar nada. O custo é o acesso aleatório, que passa a ser $O(n)$ (é preciso percorrer nó a nó).

Por que aqui: cada usuário guarda suas publicações numa lista encadeada própria. Inserir sempre no início faz a publicação mais recente aparecer primeiro *de graça*, sem precisar ordenar nada na hora de exibir. A mesma estrutura genérica (dado + destrutor) é reaproveitada nas listas de adjacência do grafo, guardando os vizinhos de cada vértice.

3.4 Fila — motor da busca em largura

Uma **fila** segue a disciplina **FIFO** (*First In, First Out*): o primeiro elemento a entrar (**enfileirar**) é o primeiro a sair (**desenfileirar**). É a estrutura de controle por trás da **busca em largura (BFS)**, detalhada na Seção 4.

Por que aqui: sem uma fila real (FIFO), não há como garantir que o grafo seja visitado *nível por nível* durante a busca de conexão e de sugestões de amizade — usar uma pilha ali, por exemplo, transformaria a busca em profundidade (DFS), que não garante encontrar primeiro os vizinhos mais próximos.

3.5 Pilha — histórico de atividades

Uma **pilha** segue a disciplina oposta, **LIFO** (*Last In, First Out*): o último elemento inserido (**empilhar**) é o primeiro a sair (**desempilhar**). Perguntar “o que aconteceu por último?” é uma operação natural em $O(1)$ numa pilha — basta olhar o topo.

Por que aqui: o histórico exige exibir as atividades da mais recente para a mais antiga — exatamente a ordem que uma pilha devolve ao ser percorrida do topo para a base. Com capacidade fixa ($N = 10$), a própria pilha descarta o elemento mais antigo (a base) a cada novo evento acima da capacidade, sem precisar de nenhuma lista auxiliar de controle.

3.6 Grafo — rede de amizades

Um **grafo** é um conjunto de **vértices** (aqui, usuários) ligados por **arestas** (aqui, amizades). Amizade é uma relação simétrica — se A é amigo de B, B é amigo de A — então o grafo é **não dirigido**.

Duas formas clássicas de representar um grafo:

- **Matriz de adjacência:** vetor $V \times V$ indicando se existe aresta entre cada par. Checar uma aresta é $O(1)$, mas o espaço é sempre $O(V^2)$, mesmo que a rede tenha poucas conexões.
- **Lista de adjacência:** cada vértice guarda apenas a lista dos seus vizinhos reais. Espaço $O(V + E)$ — muito mais eficiente quando o grafo é *esparso*, como é o caso de redes sociais reais (a maioria das pessoas não é amiga da maioria das outras).

O projeto usa lista de adjacência (reaproveitando o TAD Lista Encadeada por vértice), pelo mesmo motivo que qualquer rede social real usa: o número de amizades por pessoa é pequeno comparado ao total de usuários.

4 Busca em largura (BFS): o algoritmo por trás de duas funcionalidades

[Slide 9-10]

A **busca em largura** (*Breadth-First Search*) percorre um grafo a partir de um vértice de origem, visitando primeiro todos os vizinhos à **distância 1**, depois todos à distância 2, e assim por diante — nunca avançando para um nível mais distante antes de esgotar o nível atual. O algoritmo usa duas estruturas auxiliares:

1. Uma **fila** (FIFO) com os vértices a visitar — é o que impõe a ordem “nível por nível”: o vértice enfileirado há mais tempo é sempre o próximo a ser expandido.
2. Um vetor (ou conjunto) de **visitados**, para nunca processar o mesmo vértice duas vezes — essencial em um grafo com ciclos.

Complexidade $O(V + E)$: cada vértice entra na fila no máximo uma vez, e cada aresta é examinada no máximo uma vez (uma de cada lado).

Funcionalidade IX — verificar conexão: BFS a partir do usuário A, parando assim que o usuário B for encontrado (conectados) ou a fila esvaziar sem encontrá-lo (não conectados).

Funcionalidade X — sugestão de amizades: a mesma BFS, mas limitada a **distância 2** — ou seja, “amigos de amigos” que ainda não são amigos diretos. Para cada candidato encontrado, conta-se quantos amigos em comum ele tem com o usuário consultado (interseção das listas de adjacência dos dois), e a lista final é ordenada por

1. número de amigos em comum, decrescente;
2. em caso de empate, nome em ordem alfabética;

cortada em no máximo 3 sugestões, como pede o enunciado.

5 Requisitos não funcionais: persistência e tratamento de erros

[Slide 11]

Persistência é o princípio de que o estado de um programa não deve desaparecer quando o processo termina. Na prática, isso significa **serializar** as estruturas em memória (a árvore, as tabelas hash, o grafo, as listas, a pilha) para um formato plano em arquivo ao encerrar, e **desserializar** — reconstruir cada estrutura repovoando-a a partir do arquivo — ao iniciar. O projeto usa um formato texto simples, com seções delimitadas para usuários, amigos, publicações e histórico.

Tratamento defensivo de erros é o princípio de nunca assumir que uma entrada externa (digitada pelo usuário, ou lida de um arquivo) é válida. Toda operação da fachada retorna um código de sucesso/falha e preenche uma mensagem de erro específica para o chamador decidir o que fazer — em vez de travar o programa ou continuar com um estado inconsistente. É o princípio de “falhar graciosamente” (*fail gracefully*): o programa informa o problema e continua rodando.

6 Testes: unitário vs. integração

[Slide 12]

Um **teste unitário** verifica uma única unidade de código isoladamente — por exemplo, testar só a árvore AVL, sem depender de tabela hash, grafo ou qualquer outra parte do sistema. É o que os binários `test_avl`, `test_hash_tabela`, `test_lista`, `test_fila`, `test_pilha` e `test_grafo` fazem, cada um focado em um TAD.

Um **teste de integração** verifica se várias unidades, combinadas, produzem o comportamento esperado quando trabalham juntas — é o papel de `test_rede_social`, que exercita a fachada completa (cadastro, hash de login, grafo de amigos e pilha de histórico atuando em conjunto) do jeito que o sistema realmente é usado.

A combinação das duas camadas dá confiança em dois níveis: cada estrutura funciona corretamente por si só, e a composição delas também funciona.

7 Tabela-resumo: estrutura, operação-chave e complexidade

[Slide 4]

Estrutura	Operação-chave	Complexidade	Papel
Árvore AVL	Buscar/insérer por id	$O(\log n)$	Índice <code>id</code> -> <code>Usuario</code>
Tabela hash	Buscar/insérer por login ou nome	$O(1)$ médio	Login único; busca por nome
Lista encadeada	Inserir no início	$O(1)$	Publicações; adjacência do grafo
Fila	Enfileirar/desenfileirar	$O(1)$	Motor da BFS
Pilha	Empilhar/desempilhar	$O(1)$	Histórico (N=10 mais recentes)
Grafo (lista adj.)	BFS completa	$O(V + E)$	Conexão e sugestão de amigos

Fechamento

[Slide 13-14]

Todo esse arcabouço teórico existe para resolver um problema concreto: uma rede social com cadastro, amigos, publicações e sugestões. A partir daqui, o roteiro de demonstração ao vivo (`roteiro_demo.md`) mostra cada uma dessas estruturas em ação, com uma rede já populada de 10 usuários.